DS ASSIGNMENT - 5

1) Use linked list for polynomial representation and perform addition and subtraction operation on the polynomials.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int coeff, exp;
    struct Node *next;
};
struct Node* createNode(int c, int e) {
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->coeff = c;
    newNode->exp = e;
    newNode->next = NULL;
    return newNode;
}
void insert(struct Node **head, int c, int e) {
    struct Node *n = createNode(c, e);
    if (*head == NULL) {
        *head = n;
        return;
    }
    struct Node *t = *head;
    while (t->next != NULL)
        t = t->next;
    t->next = n;
}

void display(struct Node *p) {
```

```
    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->exp);
        p = p->next;
        if (p != NULL)
            printf(" + ");
    }
    printf("\n");
}
int main() {
    struct Node *p1 = NULL, *p2 = NULL, *sum = NULL;
    insert(&p1, 5, 2);
    insert(&p1, 4, 1);
    insert(&p1, 2, 0);
    insert(&p2, 3, 2);
    insert(&p2, 2, 1);
    insert(&p2, 1, 0);
    struct Node *a = p1, *b = p2;
    while (a != NULL && b != NULL) {
        if (a->exp == b->exp) {
            insert(&sum, a->coeff + b->coeff, a->exp);
            a = a->next;
            b = b->next;
        } else if (a->exp > b->exp) {
            insert(&sum, a->coeff, a->exp);
            a = a->next;
        } else {
            insert(&sum, b->coeff, b->exp);
            b = b->next;
        }
    }

    while (a != NULL) {
```

```
        insert(&sum, a->coeff, a->exp);
        a = a->next;
    }
    while (b != NULL) {
        insert(&sum, b->coeff, b->exp);
        b = b->next;
    }
    printf("First Polynomial: ");
    display(p1);
    printf("Second Polynomial: ");
    display(p2);
    printf("Sum: ");
    display(sum);
    return 0;
}
```

OUTPUT:

First Polynomial: 5x^2 + 4x^1 + 2x^0

Second Polynomial: 3x^2 + 2x^1 + 1x^0

Sum: 8x^2 + 6x^1 + 3x^0

2) Find the Previous Node in a Circular Linked List

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *a, *b, *c, *d;
```

```
    a = malloc(sizeof(struct Node));
    b = malloc(sizeof(struct Node));
    c = malloc(sizeof(struct Node));
    d = malloc(sizeof(struct Node));
    a->data = 10;
    b->data = 20;
    c->data = 30;
    d->data = 40;
    a->next = b;
    b->next = c;
    c->next = d;
    d->next = a;
    int key = 30;
    struct Node *temp = a;
    do {
        if (temp->next->data == key) {
            printf("Previous node of %d is %d\n",
                   key, temp->data);
            return 0;
        }
        temp = temp->next;
    } while (temp != a);
    printf("Node not found\n");
    return 0;
}
```

OUTPUT:

Previous node of 30 is 20

3) Insert at a Given Position in a Circular Linked List

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *head, *second, *third, *n;
    head = malloc(sizeof(struct Node));
    second = malloc(sizeof(struct Node));
    third = malloc(sizeof(struct Node));
    head->data = 10;
    second->data = 20;
    third->data = 30;
    head->next = second;
    second->next = third;
    third->next = head;
  int value = 15;
    int position = 2;
    n = malloc(sizeof(struct Node));
    n->data = value;
    struct Node *temp = head;
    for (int i = 1; i < position - 1; i++)
        temp = temp->next;
    n->next = temp->next;
    temp->next = n;
    printf("Circular Linked List: ");
    temp = head;

    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);
```

```
    return 0;
}
```

OUTPUT:

Circular Linked List: 10 15 20 30

4) Find the Largest Second Element in a Queue

PROGRAM:

```
#include <stdio.h>
int main() {
    int q[] = {10, 50, 30, 80, 60};
    int n = 5;
    int largest = q[0], second = -999999;
    for (int i = 1; i < n; i++) {
        if (q[i] > largest) {
            second = largest;
            largest = q[i];
        } else if (q[i] > second && q[i] < largest) {
            second = q[i];
        }
    }

    printf("Queue elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", q[i]);

    printf("\nSecond largest element: %d\n", second);
    return 0;
}
```

OUTPUT:

Queue elements: 10 50 30 80 60

Second largest element: 60

5) Insert the Third Element and Check Whether the 1st and 3rd Elements Are Equal in a Circular Linked List

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};

int main() {
    struct Node *a, *b, *c;

    a = malloc(sizeof(struct Node));
    b = malloc(sizeof(struct Node));
    c = malloc(sizeof(struct Node));
    a->data = 10;
    b->data = 20;
    c->data = 10;
    a->next = b;
    b->next = c;
    c->next = a;
    printf("First element: %d\n", a->data);
    printf("Third element: %d\n", c->data);

    if (a->data == c->data)
        printf("First and third elements are equal\n");
    else
        printf("First and third elements are not equal\n");

    return 0;
```

```
}
```

OUTPUT:

First element: 10

Third element: 10

First and third elements are equal

6) Implement Two Stacks in One Array Write routine to traverse the array and display

PROGRAM:

```
#include <stdio.h>
#define SIZE 10
int arr[SIZE];
int top1 = -1;
int top2 = SIZE;
void push1(int x) {
    if (top1 + 1 < top2)
        arr[++top1] = x;
    else
        printf("Stack Overflow\n");
}

void push2(int x) {
    if (top1 + 1 < top2)
        arr[--top2] = x;
    else
        printf("Stack Overflow\n");
}

void display() {
    printf("Stack 1: ");
    for (int i = top1; i >= 0; i--)
```

```
        printf("%d ", arr[i]);
    printf("\nStack 2: ");
    for (int i = top2; i < SIZE; i++)
        printf("%d ", arr[i]);

    printf("\n");
}

int main() {
    push1(10);
    push1(20);
    push1(30);
    push2(100);
    push2(200);
    push2(300);
    display();

    return 0;
}
```

OUTPUT:

Stack 1: 30 20 10

Stack 2: 300 200 100

7) Given a linked list containing n head nodes where every node in the linked list contains two pointers to the next node in the linked list and the head nodes are sorted in ascending order based on their data. We need to flatten the linked list so that all the nodes appear in a single linked list while maintaining sorted order.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
```

```
    struct Node *next;
    struct Node *bottom;
};

struct Node* createNode(int data) {
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->bottom = NULL;

    return newNode;
}

struct Node* merge(struct Node* a, struct Node* b) {
    if (a == NULL)
        return b;

    if (b == NULL)
        return a;
    struct Node* result;

    if (a->data <= b->data) {
        result = a;
        result->bottom = merge(a->bottom, b);
    } else {
        result = b;
        result->bottom = merge(a, b->bottom);
    }

    result->next = NULL;
    return result;
```

```
}
struct Node* flatten(struct Node* head) {
    if (head == NULL || head->next == NULL)
        return head;
    head->next = flatten(head->next);
    head = merge(head, head->next);
    return head;
}

void display(struct Node* head) {
    while (head != NULL) {
        printf("%d", head->data);
        if (head->bottom != NULL)
            printf(" -> ");
        head = head->bottom;
    }
    printf("\n");
}

int main() {
    int n, count, value;
    struct Node *head = NULL, *lastHead = NULL;
    printf("Enter number of head nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        struct Node *colHead = NULL, *last = NULL;
        printf("Enter number of nodes in column %d: ", i + 1);
        scanf("%d", &count);
        printf("Enter sorted elements: ");

        for (int j = 0; j < count; j++) {
```

```
            scanf("%d", &value);
            struct Node* newNode = createNode(value);
            if (colHead == NULL)
                colHead = newNode;
            else
                last->bottom = newNode;

            last = newNode;
        }
        if (head == NULL)
            head = colHead;
        else
            lastHead->next = colHead;
        lastHead = colHead;
    }

    head = flatten(head);
    printf("Flattened linked list: ");
    display(head);

    return 0;
}
```

OUTPUT:

```
Enter number of head nodes: 4
Enter number of nodes in column 1: 4
Enter sorted elements: 5 7 8 30
Enter number of nodes in column 2: 2
Enter sorted elements: 10 20
Enter number of nodes in column 3: 3
Enter sorted elements: 19 22 50
Enter number of nodes in column 4: 4
Enter sorted elements: 28 35 40 45
```

Flattened linked list:

5 -> 7 -> 8 -> 10 -> 19 -> 20 -> 22 -> 28 -> 30 -> 35 -> 40 -> 45 -> 50

8) Given K sorted linked lists of different sizes, we need to merge them into a single linked list while maintaining their sorted order.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node* createNode(int data) {
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;

    return newNode;
}

struct Node* mergeTwo(struct Node* a, struct Node* b) {
    if (a == NULL)
        return b;

    if (b == NULL)
        return a;

    struct Node* result;
```

```
    if (a->data <= b->data) {
        result = a;
        result->next = mergeTwo(a->next, b);
    } else {
        result = b;
        result->next = mergeTwo(a, b->next);
    }

    return result;
}

void display(struct Node* head) {
    while (head != NULL) {
        printf("%d", head->data);

        if (head->next != NULL)
            printf(" -> ");

        head = head->next;
    }

    printf("\n");
}

int main() {
    struct Node* lists[100];
    struct Node* result = NULL;
    int k, n, value;

    printf("Enter number of linked lists: ");
    scanf("%d", &k);
```

```
if (k < 1 || k > 100) {
    printf("Invalid number of lists\n");
    return 0;
}

for (int i = 0; i < k; i++) {
    struct Node *head = NULL, *tail = NULL;

    printf("Enter number of nodes in list %d: ", i + 1);
    scanf("%d", &n);

    if (n < 0) {
        printf("Invalid number of nodes\n");
        return 0;
    }

    printf("Enter sorted elements: ");

    for (int j = 0; j < n; j++) {
        scanf("%d", &value);

        struct Node* newNode = createNode(value);

        if (head == NULL)
            head = newNode;
        else
            tail->next = newNode;

        tail = newNode;
    }
```

```
        lists[i] = head;
    }

    for (int i = 0; i < k; i++)
        result = mergeTwo(result, lists[i]);

    printf("Merged linked list: ");
    display(result);

    return 0;
}
```

OUTPUT:

```
Enter number of linked lists: 3
Enter number of nodes in list 1: 3
Enter sorted elements: 1 4 7
Enter number of nodes in list 2: 2
Enter sorted elements: 2 5
Enter number of nodes in list 3: 3
Enter sorted elements: 3 6 8
Merged linked list: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8
```

9) Swap the Kth node from the beginning with the Kth node from the end in a singly linked list.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
```

```
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;

    return newNode;
}

void insert(struct Node** head, int data) {
    struct Node* newNode = createNode(data);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

void swapKth(struct Node** head, int k) {
    int n = 0;
    struct Node* temp = *head;

    while (temp != NULL) {
        n++;
        temp = temp->next;
```

```
}

if (k <= 0 || k > n) {
    printf("Invalid K value\n");
    return;
}

if (2 * k - 1 == n)
    return;

struct Node *prev1 = NULL, *curr1 = *head;
struct Node *prev2 = NULL, *curr2 = *head;

for (int i = 1; i < k; i++) {
    prev1 = curr1;
    curr1 = curr1->next;
}

for (int i = 1; i < n - k + 1; i++) {
    prev2 = curr2;
    curr2 = curr2->next;
}

/* If curr1 is immediately before curr2 */
if (curr1->next == curr2) {
    if (prev1 != NULL)
        prev1->next = curr2;
    else
        *head = curr2;

    curr1->next = curr2->next;
    curr2->next = curr1;
```

```
        return;
    }

    /* If curr2 is immediately before curr1 */
    if (curr2->next == curr1) {
        if (prev2 != NULL)
            prev2->next = curr1;
        else
            *head = curr1;

        curr2->next = curr1->next;
        curr1->next = curr2;
        return;
    }

    if (prev1 != NULL)
        prev1->next = curr2;
    else
        *head = curr2;

    if (prev2 != NULL)
        prev2->next = curr1;
    else
        *head = curr1;

    temp = curr1->next;
    curr1->next = curr2->next;
    curr2->next = temp;
}

void display(struct Node* head) {
    while (head != NULL) {
```

```
        printf("%d", head->data);

        if (head->next != NULL)
            printf(" -> ");

        head = head->next;
    }

    printf("\n");
}

int main() {
    struct Node* head = NULL;
    int n, value, k;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid number of nodes\n");
        return 0;
    }

    printf("Enter elements: ");

    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        insert(&head, value);
    }

    printf("Enter K: ");
    scanf("%d", &k);
```

```
    printf("Original list: ");
    display(head);

    if (k <= 0 || k > n) {
        printf("Invalid K value\n");
        return 0;
    }

    swapKth(&head, k);

    printf("After swapping: ");
    display(head);

    return 0;
}
```

OUTPUT:

Enter number of nodes: 5

Enter elements: 10 20 30 40 50

Enter K: 2

Original list: 10 -> 20 -> 30 -> 40 -> 50

After swapping: 10 -> 40 -> 30 -> 20 -> 50

10) Multiply two numbers represented by two linked lists, where each node contains one digit of the number.

PROGRAM:

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};
```

```
struct Node* createNode(int data) {
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;
    newNode->next = NULL;

    return newNode;
}

void insert(struct Node** head, int digit) {
    struct Node* newNode = createNode(digit);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

long long toNumber(struct Node* head) {
    long long number = 0;

    while (head != NULL) {
        number = number * 10 + head->data;
```

```
        head = head->next;
    }

    return number;
}

void display(struct Node* head) {
    while (head != NULL) {
        printf("%d", head->data);

        if (head->next != NULL)
            printf(" -> ");

        head = head->next;
    }

    printf("\n");
}

int main() {
    struct Node *num1 = NULL, *num2 = NULL;
    int n, digit;

    printf("Enter number of digits in first number: ");
    scanf("%d", &n);

    if (n <= 0) {
        printf("Invalid input\n");
        return 0;
    }

    printf("Enter digits from left to right: ");
```

```
for (int i = 0; i < n; i++) {
    scanf("%d", &digit);

    if (digit < 0 || digit > 9) {
        printf("Invalid digit\n");
        return 0;
    }

    insert(&num1, digit);
}

printf("Enter number of digits in second number: ");
scanf("%d", &n);

if (n <= 0) {
    printf("Invalid input\n");
    return 0;
}

printf("Enter digits from left to right: ");

for (int i = 0; i < n; i++) {
    scanf("%d", &digit);

    if (digit < 0 || digit > 9) {
        printf("Invalid digit\n");
        return 0;
    }

    insert(&num2, digit);
}
```

```
    printf("First number: ");
    display(num1);

    printf("Second number: ");
    display(num2);

    long long a = toNumber(num1);
    long long b = toNumber(num2);

    printf("Product = %lld\n", a * b);

    return 0;
}
```

OUTPUT:

```
Enter number of digits in first number: 3
Enter digits from left to right: 1 2 3
Enter number of digits in second number: 2
Enter digits from left to right: 4 5
First number: 1 -> 2 -> 3
Second number: 4 -> 5
Product = 5535
```